



touying-ufac

Touying slide theme for the Federal University of Acre (UFAC)

v0.1.0 September 18, 2026

<https://github.com/lucaslrodri/touying-ufac-unofficial>

Lucas Lima Rodrigues

A **Touying** theme for lecture slides, following the visual identity of the **Federal University of Acre**. Slides are written with headings and a few Markdown-like shorthands for boxes, quotations, arrow lists and emphases. This manual is set with the rules of the theme itself.

Contents

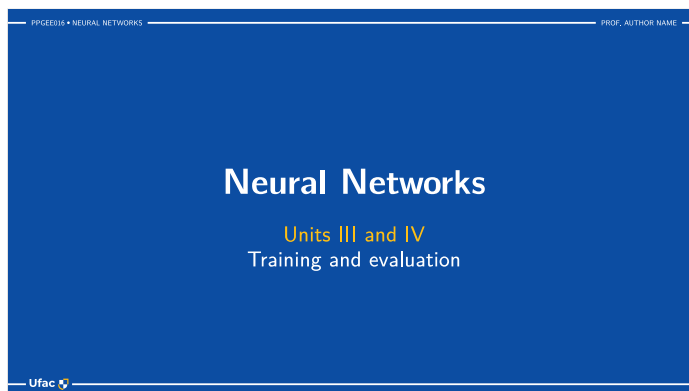
1 Quick start	2	8.1 Tables	24
1.1 The information of the deck	2	8.2 Figures and math	24
1.2 The theme and the cover	3	8.3 Code	24
1.3 Language	4	8.4 Icons	26
1.4 Dependencies	5	9 Touying features	29
2 Slides and headings	6	10 Index of definitions	31
2.1 Sections	6		
2.2 Slides and continuations	6		
2.3 Exercises and examples	7		
2.4 Subtitles	9		
2.5 Slide functions	10		
3 Inline formatting and shorthands	14		
3.1 Bold, italic and alert	14		
3.2 Highlight and underline	14		
3.3 Code chips	14		
3.4 Colors by context	15		
3.5 Colored emphases	15		
3.6 Escapes	16		
4 Lists and terms	17		
4.1 Arrow lists	17		
4.2 Bullet lists	17		
4.3 Numbered lists	17		
4.4 Terms	18		
5 Boxes and quotes	19		
5.1 Emphasis boxes	19		
5.2 The box shorthand	19		
5.3 Equation boxes	20		
5.4 Quotations	21		
6 Columns	22		
7 Colors and logo	23		
8 Tables, figures, math, code and icons	24		

1 Quick start

A deck imports Touying and the theme, applies `ufac-theme` and usually starts with the cover:

```
1 #import "@preview/touying:0.7.4": *
2 #import "@preview/touying-ufac:0.1.0": *
```

```
1 #show: ufac-theme.with(
2   aspect-ratio: "16-9",
3   lang: "en",
4   config-info(
5     subject: [Neural
6       Networks],
7     subtitle: [Units III and
8       IV],
9     title: [Training and
10      evaluation],
11     author: [Prof. Author
12      Name],
13     subject-code: [PPGEE016],
14     counter-prefix: [1.],
15   ),
16 )
17 #title-slide()
```



`typst init @preview/touying-ufac:0.1.0` creates a deck from the package template, and `example/main.typ` shows the elements of the syntax in a short deck. From here on, every example shows the source on the left and the result on the right: a real slide when the feature belongs to the slide (headers, pills, continuations), otherwise the content alone, laid out by the same rules as in a deck.

1.1 The information of the deck

`config-info` is Touying's; the theme reads these fields:

Field	Where it appears
<code>subject</code>	first line of the cover (large, white) and left side of the footer
<code>subtitle</code>	second line of the cover, in yellow
<code>title</code>	third line of the cover, in white
<code>author</code>	top right of the cover and right side of the footer
<code>subject-code</code>	before the subject in the footer: "CODE · SUBJECT"
<code>counter-prefix</code>	prefix of the exercise and example numbers: <code>[1.]</code> gives "Exercise 1.N", <code>none</code> gives "Exercise N"

1.2 The theme and the cover

ufac-theme()

```
ufac-theme(  
  aspect-ratio: "16-9" str  
  lang: "pt-br" str  
  exercise-name: auto auto content  
  example-name: auto auto content  
  part-name: auto auto content  
  footer-left: auto auto content function  
  footer-right: auto auto content function  
  ..args dictionary  
  body content  
) -> content
```

The theme. It sets the page (22pt New Computer Modern Sans, the UFAC header and footer) and every rule of the syntax, and takes Touying's configs as extra arguments.

```
1 #show: ufac-theme.with(  
2   aspect-ratio: "16-9",  
3   config-info(  
4     title: [Training and evaluation], subtitle: [Units III and IV],  
5     author: [Prof. Name], subject: [Neural Networks], subject-code: [PPGEE016],  
6     counter-prefix: [1.],  
7   ),  
8 )
```

`config-info` carries `subject`, `subtitle`, `title`, `author`, `subject-code` and `counter-prefix`, the prefix of the exercise and example numbers ("Exercise 1.N"; `none` gives "Exercise N").

Argument

`aspect-ratio: "16-9" str`

Any ratio accepted by Touying's `utils.page-args-from-aspect-ratio`: "16-9", "4-3", ...

Argument

`lang: "pt-br" str`

Language of the deck: "pt-br" (or "pt"), "en" or "es", optionally with a region ("en-US"). It sets `text.lang/region` and the localized names below.

Argument

`exercise-name: auto auto | content`

Name in the pill of exercise slides; `auto` is Exercício / Exercise / Ejercicio.

Argument

`example-name: auto auto | content`

Name in the pill of example slides; `auto` is Exemplo / Example / Ejemplo.

Argument

`part-name: auto auto | content`

Prefix of the section slide ("Part N:"); `auto` is Parte / Part / Parte.

Argument

footer-left: **auto**

auto | **content** | **function**

Left text of the footer; a function receives `self`. `auto` is the subject code, a dot and the subject (the dot only when both are given).

Argument

footer-right: **auto**

auto | **content** | **function**

Right text of the footer; a function receives `self`. `auto` is the author.

Argument

..args

dictionary

Touying configs, typically `config-info(..)`; also `config-common(..)`, `config-page(..)`.

Argument

body

content

The deck.

title-slide()

```
title-slide(  
  ) -> content
```

Cover: blue page with the subject (large, white), the subtitle (yellow) and the title (white) of `config-info` centered, `footer-left`/`footer-right` in white at the top and the white logo at the bottom. The slide counter is frozen, so the cover does not count.

1.3 Language

The theme speaks Portuguese by default (`lang: "pt-br"`): Exercício, Exemplo, Parte. `lang: "en"` or `"es"` switches the text language and those three names, and each name can still be overridden.

```
1  #show: ufac-theme.with(  
2    lang: "es",  
3    part-name: [Unidad],  
4    config-info(subject:  
5      [Redes], author: [Autor]),  
6  )  
7  = Redes multicapa  
8  
9  == Perceptrón <exercise>  
10  
11 Entrene un perceptrón para la  
   puerta AND.
```

Unidad 1: Redes multicapa

Ejercicio 1.1 (Perceptrón)

Entrene un perceptrón para la puerta AND.

 REDES
 AUTOR 2

1.4 Dependencies

The versions below are read from the sources of the package, so this table cannot go stale. All of them are the latest releases at the time of writing and are fetched automatically by Typst.

Package	Version	Role
<code>codly</code>	1.3.0	code blocks; its functions are re-exported by the theme
<code>linguify</code>	0.5.0	localized names (Exercise, Example, Part)
<code>touying</code>	0.7.4	the slide engine; the deck imports it too, with the same version

Besides the packages, the theme needs **Typst 0.14.0 or later** and the font **New Computer Modern Sans** installed on the system (it is not bundled with Typst, unlike **New Computer Modern Math**, used for the equations). Building this manual also takes [tidy](#) 0.4.3, and the test suite runs on [Typanic](#) 0.4.1.

2 Slides and headings

Headings drive the deck: no function call is needed for the ordinary slides.

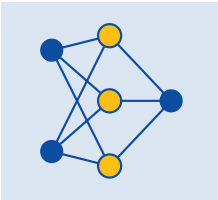
Markup	Effect
<code>= Title</code>	section slide “Part N: Title”; what follows the heading goes to the right
<code>== Title</code>	new slide with a blue header, fitted to the width
<code>== Title <exercise></code>	exercise slide: red pill “Exercise 1.N (Title)”, numbered automatically
<code>== Title <example></code>	example slide: green pill “Example 1.N (Title)”, with its own counter
<code>=== Subtitle</code>	blue pill; no new slide; repeated at the top of every continuation page
<code>==== Subtitle</code>	the same pill, not repeated
<code>---</code>	alone on a line: next page of the same slide (same header, same number)

2.1 Sections

The content right after a `=` heading, typically an image, sits on the right, flush with the page edge. `#pad(right: 1.8em)[#image(...)]` restores the page margin.

```
1  = Multilayer networks
2  #image("figure.svg")
```

Part 1:
Multilayer networks



Ufac PPGED10 • NEURAL NETWORKS PROF. AUTHOR NAME 1

2.2 Slides and continuations

```
1  == Gradient descent
2
3  The weights follow the
4  negative gradient of the
5  error,
6  one step per batch.
7  ---
8  The same slide goes on: same
9  header, same number.
```

Gradient descent

The weights follow the negative gradient of the error, one step per batch.

Ufac PPGED10 • NEURAL NETWORKS PROF. AUTHOR NAME 1

Gradient descent

The same slide goes on: same header, same number.

Ufac
PPGEED16 • NEURAL NETWORKS
PROF. AUTHOR NAME
2

Caution

Touying breaks the page at *any* top-level em dash, so an em dash next to inline markup, as in `Text – *bold*`, also starts a new page. Use an en dash or wrap the paragraph in `#[ ]`.

2.3 Exercises and examples

A label on the `==` heading turns the header into a numbered pill. Any suffix gives the same slide and makes the label unique, so the slide can be cited: `@exercise-ogata` becomes “Exercise 1.N”, bold, in the color of the kind and linked to the slide. `#exercise-slide` and `#example-slide` are the function forms, with a `color` option; they cannot be cited.

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15

```

== Ogata 4.6 <exercise-ogata>
Find the transfer function of
the system.
== XOR network <example>
Two hidden neurons are
enough.
#exercise-slide(title:
[Haykin 4.2], color:
colors.primary)[
The function form, with
another color.
]
== Review
Solve @exercise-ogata before
the next class.

```

Exercise 1.1 (Ogata 4.6)
Find the transfer function of the system.

Example 1.1 (XOR network)
Two hidden neurons are enough.

Ufac
PPGEED16 • NEURAL NETWORKS
PROF. AUTHOR NAME
1

Ufac
PPGEED16 • NEURAL NETWORKS
PROF. AUTHOR NAME
2

Exercise 1.2 (Haykin 4.2)

The function form, with another color.

Ufac
PPGE018 • NEURAL NETWORKS
PROF. AUTHOR NAME
3

Review

Solve Exercise 1.1 before the next class.

Ufac
PPGE018 • NEURAL NETWORKS
PROF. AUTHOR NAME
4

exercise-slide()

```
exercise-slide(
  title: [] content
  color: auto auto color
  body content
) -> content
```

Exercise slide: the header becomes a pill “Exercise 1.N (Title)”, numbered automatically. The function form of `(= Title <exercise>)`. Only the heading form can carry a unique label `(= Title <exercise-ogata>)` and be cited with `@exercise-ogata`, which shows “Exercise 1.N” linked to the slide.

```
1 #exercise-slide(title: [ogata 4.6])[Find the transfer function.]
```

Argument

title: [] content

Text in parentheses after the number; empty omits the parentheses.

Argument

color: auto auto | color

Color of the pill; `auto` is red (`tertiary`).

Argument

body content

Content of the slide; `---` inside it continues the same exercise.

example-slide()

```
example-slide(  
  title: [] content  
  color: auto auto color  
  body content  
) -> content
```

Example slide: a pill “Example 1.N (Title)”, with a counter independent from the exercises. The function form of `== Title <example>`; see `exercise-slide()`.

```
1 #example-slide(title: [XOR network])[Two hidden neurons are enough.]
```

Argument

title: []

content

Text in parentheses after the number; empty omits the parentheses.

Argument

color: auto

auto | color

Color of the pill; `auto` is green (`quaternary`).

Argument

body

content

Content of the slide.

2.4 Subtitles

A `===` subtitle is repeated on the pages that continue the slide, unless the continuation starts with another `===`. That holds for `---`, for `#pagebreak()` and for content that overflows by itself.

Activation functions

Sigmoid

Smooth and bounded, but it saturates.

Derivative

Not repeated on the next page.

Ufac  PPGE010 • NEURAL NETWORKS

PROF. AUTHOR NAME

1

1

2

3

4

5

6

7

8

9

10

11

12

13

== Activation functions

=== Sigmoid

Smooth and bounded, but it saturates.

==== Derivative

Not repeated on the next page.

The `===` pill is repeated here.

Activation functions

Sigmoid

The === pill is repeated here.

Ufac

PPGEE018 • NEURAL NETWORKS

PROF. AUTHOR NAME

2

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

== Training loop

=== Steps of one epoch

+ Shuffle the training set

+ Split it into batches

+ Forward pass on the batch

+ Compute the loss

+ Backward pass

+ Update the weights

+ Repeat for every batch

+ Evaluate on the validation set

+ Save the best weights

+ Check the stopping rule

+ Decay the learning rate

+ Log the metrics

+ Plot the learning curves

+ Compare with the baseline

+ Start the next epoch

Training loop

Steps of one epoch

1. Shuffle the training set
2. Split it into batches
3. Forward pass on the batch
4. Compute the loss
5. Backward pass
6. Update the weights
7. Repeat for every batch
8. Evaluate on the validation set
9. Save the best weights
10. Check the stopping rule
11. Decay the learning rate
12. Log the metrics

Ufac

PPGEE018 • NEURAL NETWORKS

PROF. AUTHOR NAME

1

Training loop

Steps of one epoch

13. Plot the learning curves
14. Compare with the baseline
15. Start the next epoch

Ufac

PPGEE018 • NEURAL NETWORKS

PROF. AUTHOR NAME

2

Only a top-level `===` is repeated; inside `#[ ]`, a column or a box the pill stays in the flow. `config-common(breakable: false)` makes Touying warn about overflow instead.

2.5 Slide functions

`#slide` is the slide that every `==` opens; calling it gives access to Touying's arguments, such as `composer`. `#empty-slide` drops the header and keeps the footer. `#title-slide()` is the cover, shown in Section 1.

```

1 #slide(composer: (1fr, 2fr))[
2   Left body
3 ][
4   Right body, twice as wide.
5 ]
6
7 #empty-slide[
8   #align(center + horizon)[No
9   header, footer kept.]

```

Left body

Right body, twice as wide.

Ufac • PPGEED18 • NEURAL NETWORKS • PROF. AUTHOR NAME • 1

No header, footer kept.

Ufac • PPGEED18 • NEURAL NETWORKS • PROF. AUTHOR NAME • 2

slide()

```

slide(
  config: (:) dictionary
  repeat: auto auto int
  setting: body => body function
  composer: auto auto array function
  ..bodies content
) -> content

```

Default slide of the theme, with header and footer. Every `==` heading opens one, so it is rarely called by hand; the explicit form gives access to Touying's arguments.

A top-level `==` subtitle is repeated at the top of every continuation page of the slide, whether the page comes from `---`, from `#pagebreak()` or from content that overflows by itself, unless the continuation starts with another `==`. A subtitle nested in `#[ ]`, a column or a box is not repeated.

```
1 #slide(composer: (1fr, 2fr))[Left][Right]
```

Argument

config: (:) dictionary

Slide configuration: a `config-xxx` dictionary (merge several with `utils.merge-dicts`).

Argument

repeat: auto auto | int

Number of subslices; `auto` lets Touying count the animations.

Argument

setting: `body => body`

function

Set/show rules applied to the slide body.

Argument

composer: `auto`

`auto` | `array` | `function`

Layout of the bodies, e.g. `(1fr, 2fr)`, as in Touying's `side-by-side`.

Argument

`..bodies`

content

The slide bodies.

empty-slide()

```
empty-slide(  
  config: (:) dictionary  
  repeat: auto auto int  
  setting: body => body function  
  composer: auto auto array function  
  ..bodies content  
) -> content
```

Slide without the header but with the footer (Touying's `empty-slide` removes both). Same arguments as `slide()`.

Argument

config: `(:)`

dictionary

Slide configuration, see `slide.config`.

Argument

repeat: `auto`

`auto` | `int`

Number of subslides.

Argument

setting: `body => body`

function

Set/show rules applied to the slide body.

Argument

composer: `auto`

`auto` | `array` | `function`

Layout of the bodies.

Argument

`..bodies`

content

The slide bodies.

new-section-slide()

```
new-section-slide(  
  config: (:) dictionary  
  level: 1 int  
  numbered: true bool  
  body content  
) -> content
```

Section slide, created by every `= Title`: “Part N: Title” in blue on the left and the content right after the `=` (typically an `#image`) on the right, flush with the page edge. `#pad(right: 1.8em)[#image(...)]` restores the page margin. Not meant to be called by hand.

Argument

`config: (:)` dictionary
Slide configuration, see `slide.config`.

Argument

`level: 1` int
Heading level, passed by Touying.

Argument

`numbered: true` bool
Passed by Touying; the part is always numbered.

Argument

`body` content
Content that follows the `=` heading.

3 Inline formatting and shorthands

Most emphases have two forms: a function and a **shorthand**, a short piece of markup that the theme rewrites into the same function. The shorthands take plain text only (no markup inside) and none of them applies inside `raw`.

Function	Shorthand	Result
<code>#strong[text]</code>	<code>*text*</code>	black bold
<code>#emph[text]</code>	<code>_text_</code>	black italic
<code>#alert[text]</code> , <code>#primary[text]</code>	<code>~~text~~</code>	blue bold
<code>#highlight[text]</code>	<code>==text==</code>	light-yellow background
<code>#underline[text]</code>	<code>~text~</code>	strong yellow underline
<code>#secondary</code> , <code>#tertiary</code> , <code>#quaternary</code>		bold in yellow, red, green
<code>#emph-box[..]</code>	<code>> text</code>	box, Section 5
<code>\$ #eq-box[\$..\$] \$</code>	<code>> \$.. \$</code>	equation box, Section 5
<code>#quote-box[..]</code>	<code>"text"</code>	quotation, Section 5
<code>#arrows[..]</code>	<code>-> item</code>	arrow list, Section 4

3.1 Bold, italic and alert

```
1 Black *bold* and _italic_.
```

```
2
```

```
3 Blue #alert[alert], the same as
```

```
4 ~~this shorthand~~.
```

Black **bold** and *italic*.

Blue **alert**, the same as **this shorthand**.

3.2 Highlight and underline

The `==text==` shorthand cannot open a line (there it is a heading) and its text neither starts nor ends with a space; `#highlight` has no such limits and takes another fill, such as the `-lighter` tones of the palette.

```
1 A ==highlighted== word and a
2 #highlight(fill: colors.quaternary-
3   lighter)[green one].
```

```
4 A ~strong underline~ and
5 #underline[the function form]; `Fig.~1`
6 keeps
7 its non-breaking space.
```

A **highlighted** word and a **green one**.

A **strong underline** and the function form

`Fig.~1` keeps its non-breaking space.

3.3 Code chips

Inline `raw` is a gray chip. Inside a title or a box it follows the color of the container, and inside a pill it is inverted.

```
1 Call `fit(x, y)` first.
2
3 #emph-box(color: colors.tertiary)[
4   Never call `predict` before `fit`.
5 ]
```

Call `fit(x, y)` first.

Never call `predict` before `fit`.

3.4 Colors by context

Bold text, alerts, highlights, underlines, arrows, chips and icons read the color of what contains them: the box color inside an `emph-box`, white or yellow inside a pill, where the pill color would hide them.

```
1 == Header with `raw` and
   *bold*
2
3 === Pill: *bold*, ~~alert~~,
   `raw`, #icon("check")
4
5 #emph-box(title: [Title:
   *bold*, ~~alert~~ and `raw`])
6   In the body: *bold*, `raw`
   and #icon("check").
7 ]
```

Header with `raw` and bold

Pill: bold, alert, raw ✓

Title: bold, alert and raw

In the body: bold, raw and ✓.

Ufac • PPGEDD • NEURAL NETWORKS • PROF. AUTHOR NAME • 1

3.5 Colored emphases

Bold text named after the color families of Section 7. A typical use is a label that opens a sentence.

primary()

```
primary(
  body content
) -> content
```

Bold text in UFAC blue: the same as `#alert` (Touying's method) and as the `~~text~~` shorthand. Inside a pill (a `===` subtitle, an exercise header, the title of an `emph-box()`) it turns UFAC yellow, where blue would vanish.

```
1 A #primary[definition], the same as
   ~~this shorthand~~.
```

A definition, the same as this shorthand.

Argument

body content

Text to emphasize.

secondary()

```
secondary(
  body content
) -> content
```

Bold text in UFAC yellow. It is weak on white: meant for short labels and dark backgrounds.

```
1 A #secondary[remark] in yellow.
```

A remark in yellow.

Argument

body content

Text to emphasize.

tertiary()

```
tertiary(  
  body content  
) -> content
```

Bold text in red, for warnings and problems.

1 `#tertiary`[Caution:] the gradient vanishes.

Caution: the gradient vanishes.

Argument

body

content

Text to emphasize.

quaternary()

```
quaternary(  
  body content  
) -> content
```

Bold text in green, for advantages and correct answers.

1 `#quaternary`[Correct:] the series converges.

Correct: the series converges.

Argument

body

content

Text to emphasize.

3.6 Escapes

A backslash on the sign keeps a shorthand literal. An escaped `>` is only recognized when a word follows it.

1 `\==not highlighted==` and `\~not underlined\~`

2

3 `\>` not a box

4

5 `-\>` not a list

6

7 `\"not a quotation\"`

`==not highlighted==` and `~not underlined~`

`>` not a box

`->` not a list

`"not a quotation"`

4 Lists and terms

4.1 Arrow lists

Consecutive lines starting with `->` become one list with a blue arrow. The list has to start the paragraph: a blank line comes before the first `->`. An arrow in the middle of a line and the math arrow are left alone.

1	<code>-> consequence</code>	$\rightarrow$ consequence
2	<code>-> another consequence</code>	$\rightarrow$ another consequence
3		
4	Here a <code>-></code> b stays text, and <code>\$a -> b\$</code> is math.	Here a <code>-></code> b stays text, and $a \rightarrow b$ is math.

arrows()

```
arrows(  
  color: colors.primary color  
  body content  
) -> content
```

Arrow lists with the arrow in another color, for everything inside the body. The `-> item` shorthand itself needs no function: consecutive lines starting with `->` become one list with a blue arrow (the box color inside an `emph-box()`). The list must start the paragraph, and `-\> item` escapes it.

1	<code>#arrows(color: colors.quaternary)[</code>	$\rightarrow$ green here
2	<code>-> green here</code>	$\rightarrow$ and here
3	<code>-> and here</code>	
4	<code>]</code>	

Argument

color: colors.primary
Color of the arrows.

color

Argument

body
Content with `-> item` paragraphs.

content

4.2 Bullet lists

Three levels of yellow markers: a filled square, a rotated hollow square and a small circle.

1	<code>- first level</code>	■ first level
2	<code>- second level</code>	▣ second level
3	<code>- third level</code>	● third level
4	<code>- first level again</code>	■ first level again

4.3 Numbered lists

Numbers are always blue and bold: `1.`, `a)`, `i.` by level. `#set enum(numbering: "a")` starts at `a)` instead and keeps the colors. Typst only knows `+` and `1.` as enum markers in markup, so `a)` at the start of a line is text.

1	+ first	1. first
2	+ nested	a) nested
3	+ deeper	i. deeper
4	+ second	2. second
5		
6	#[	a) starts at a)
7	#set enum(numbering: "a")	i. then i.
8	+ starts at a)	
9	+ then i.	
10	]	

4.4 Terms

1	/ Epoch: one pass over the training set.	Epoch: one pass over the training set.
2	/ Batch: the samples of one update.	Batch: the samples of one update.

5 Boxes and quotes

5.1 Emphasis boxes

emph-box()

```
emph-box(  
  color: colors.primary color  
  title: none none content  
  body content  
) -> content
```

Emphasis box. Without a title it has a thin border; with a title, a badge sits over the top border. Inside the box the `-` markers, the `->` arrow, `*bold*`, `==highlight==` and `#underline` take the box color; an explicit `#arrows(color:)`, `#highlight(fill:)` or `#underline(stroke:)` wins. Colors by convention: primary for definitions, quaternary for advantages, tertiary for problems, secondary for remarks.

```
1 #emph-box(color: colors.quaternary,  
2   title: [Pros])  
3   - Fast and *simple*  
4   - Works with ==highlights==  
5 ]
```

Pros

- Fast and **simple**
- Works with highlights

Shorthand, always in the primary color: a paragraph whose lines start with `>`; a first line `> == Title` gives the title. Only inline content fits the shorthand. For a framed equation see `eq-box()`.

Argument

color: colors.primary

color

Color of the border and badge, and of the bold text, markers, arrows and highlights inside.

Argument

title: none

none | content

Badge over the top border; `none` gives the thin-border box.

Argument

body

content

Content of the box.

```
1 #emph-box[A definition, in the primary  
2   color.]  
3  
4 #emph-box(color: colors.tertiary,  
5   title: [Problem])  
6   The gradient *vanishes* and ~stalls~.  
7   -> the arrow is red too  
8 ]
```

A definition, in the primary color.

Problem

The gradient **vanishes** and stalls.
→ the arrow is red too

5.2 The box shorthand

A paragraph whose lines start with `>` is a box in the primary color; a first line `> == Title` gives the title.

```

1 > A box without title, with *bold* and
  $x^2$.
2
3 > == Definition
4 > A box with a title.
5 > The second line flows on.

```

A box without title, with **bold** and x^2 .

Definition

A box with a title. The second line flows on.

Limits of the shorthand

Only inline content fits: lists and code blocks need `#emph-box`, and `>` before a block equation is an `eq-box`. A loose `>` at the start of a paragraph is read as a box: write `$a > b$`, or escape it (Section 3.6).

5.3 Equation boxes

eq-box()

```

eq-box(
  color: auto auto color
  body content
) -> content

```

Equation box: a light frame around a key equation (the `key-eq` helper of the lecture decks). It goes **inside** the equation, like the `boxeq` of the bookly package, so the equation keeps its centering, spacing and numbering, and a part of an equation can be framed too. The body is written as math, between dollar signs; it is set in display style whatever the spacing around it. A box around the whole equation is padded generously, a box around a part of one tightly, and in both cases the frame holds tall fractions and the limits of sums.

It also works in a line of text or in an inline equation: there the body is in text style and the box does not change the line spacing.

```

1 The output #eq-box[$y = phi(v)$] and
  the
2 mean #eq-box[$1/N sum_i y_i$].

```

The output $y = \varphi(v)$ and the mean $\frac{1}{N} \sum_i y_i$.

```

1 $ #eq-box[$v = bold(w)^top bold(x)$] $
2
3 $ y = #eq-box(color: colors.quaternary)
  [$phi(v)$] + b $

```

$$v = w^\top x$$

$$y = \varphi(v) + b$$

Shorthand, always in the primary color: `>` in front of a block equation, on the same line or alone on the line before it.

```

1 > $ E = 1/(2N) sum_i (y_i - hat(y)_i)^2
  $

```

$$E = \frac{1}{2N} \sum_i (y_i - \hat{y}_i)^2$$

Argument

color: auto

auto | color

Base color of the fill and border; `auto` is blue, or the box color inside an `emph-box()`.

Argument

body

content

The equation, as math: $[x = 2]$ or $(x = 2)$.

Why the dollar signs inside

A content block inside an equation is markup, not math: `#eq-box[x = 2]` would be the upright text “x=2”, and an underscore or an asterisk in it would not even parse. `eq-box` refuses such a body with a hint. `> $x$`, with an inline equation, is an ordinary box: the shorthand takes a block equation.

5.4 Quotations

quote-box()

```
quote-box(  
  color: colors.secondary color  
  body content  
) -> content
```

Quotation box: a vertical bar on the left. Typst’s `#quote(attribution: [Author])[...]` is shown as a `quote-box` too, with the attribution on the right.

```
1 #quote-box(color: colors.primary)[  
2   All models are wrong, but some are  
3   useful.  
3 ]
```

All models are wrong, but some are useful.

Shorthand: a paragraph made only of “quoted” runs, one per line; each run becomes a box and the colors cycle secondary, primary, quaternary, tertiary. Quotes inside a sentence are left alone.

Argument

color: colors.secondary

color

Color of the bar.

Argument

body

content

The quotation.

```
1 "All models are wrong"  
2 "but some are useful"  
3  
4 A sentence with a "quotation" inside.  
5  
6 #quote(attribution: [George Box])[  
7   All models are wrong.  
8 ]
```

All models are wrong

but some are useful

A sentence with a “quotation” inside.

All models are wrong.

— George Box

6 Columns

cols()

```
cols(  
  columns: auto auto array  
  gutter: 1em length  
  divider: false bool  
  ..bodies content  
) -> content
```

Columns: a thin wrapper over Touying's `cols` (a `grid`), with an optional divider. Touying's own `side-by-side` (no divider) is still available.

```
1 #cols(divider: true, columns: (1fr,  
2   2fr))[  
3   Left  
4 ] [  
5   -> the shorthands work  
6   -> inside a column  
7 ]
```

Left

→ the shorthands work
→ inside a column

Argument

columns: auto auto array

Column widths, as Touying accepts them: `auto` (equal widths) or an array such as `(1fr, 2fr)`.

Argument

gutter: 1em length

Space between the columns.

Argument

divider: false bool

A blue vertical bar (2.2pt) between the columns; the gutter is then used as inset.

Argument

..bodies content

One content block per column.

```
1 #cols[Left][Middle][Right]
```

Left

Middle

Right

Touying's `side-by-side` (no divider) is still available, and `#slide(composer: (1fr, 2fr))[...][...]` lays out the bodies of a whole slide.

7 Colors and logo

Five families of seven tones. The light tones serve as fills of highlights and boxes, the dark ones as text on white.

	-darkest	-darker	-dark	base	-light	-lighter	-lightest
primary	#031329	#052349	#083671	#0C4DA2	#5582BE	#9EB8DA	#DBE4F1
secondary	#403005	#735609	#B2860E	#FFBF14	#FFD25A	#FFE5A1	#FFF5DC
tertiary	#30070B	#560D14	#851520	#BE1E2D	#D2626C	#E5A5AB	#F5DDE0
quaternary	#022C20	#044F3A	#067B5A	#09B081	#53C8A7	#9DDFCD	#DAF3EC
neutral	#000000	#2B2B2B	#555555	#808080	#AAAAAA	#D5D5D5	#FFFFFF

The tones are derived from the base in RGB: `darken(75%)`, `darken(55%)`, `darken(30%)`, the base, `lighten(30%)`, `lighten(60%)` and `lighten(85%)`. The neutral family follows Touying's default instead, from black to white.

```
1 #highlight(fill: colors.tertiary-  
  lighter)[red fill]  
2 and #text(fill: colors.primary-dark)  
  [dark blue text].
```

red fill and dark blue text.

colors

dictionary

The theme palette: the official UFAC colors plus a neutral gray, named after Touying's `config-colors` keys (`primary`, `secondary`, `tertiary`, `neutral`) plus `quaternary`. Each family has 7 tones, `<family>-darkest`, `-darker`, `-dark`, the base, `-light`, `-lighter` and `-lightest`, so `colors.primary-lighter`, `colors.quaternary-dark` etc. exist. The whole dictionary is passed to `config-colors`, hence the same keys are available as `self.colors.*`.

Family	Base	Role
primary	#0C4DA2	titles, <code>-></code> markers, tables, <code>#alert</code> / <code>#primary</code> , pills
secondary	#FFBF14	<code>-</code> markers, <code>#secondary</code> , quotes; <code>-lighter</code> is the highlight
tertiary	#BE1E2D	<code>#tertiary</code> , exercises
quaternary	#09B081	<code>#quaternary</code> , examples
neutral	#808080	<code>-darkest</code> is the bold/italic text, <code>-lightest</code> the background

ufac-logo

str

Official UFAC logo as an SVG string (yellow shield and blue lettering), rendered in the footer with `image(bytes(ufac-logo), height: 0.64em)`. The title slide replaces the blue `#0c4da2` with white.

```
1 #image(bytes(ufac-logo), height: 2em)
```

Ufac 

8 Tables, figures, math, code and icons

8.1 Tables

The header row is blue with white text, and formulas in it turn white too.

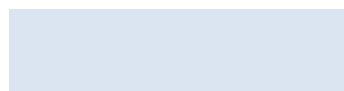
```
1 #table(  
2   columns: 3,  
3   table.header([Model], [Layers],  
4     [$eta$]),  
5   [MLP], [3], [0.1],  
6   [CNN], [8], [0.01],  
7 )
```

Model	Layers	η
MLP	3	0.1
CNN	8	0.01

8.2 Figures and math

A figure caption shows its body only, without “Figure N”. Math uses New Computer Modern Math, and block equations sit closer to the text than Typst’s default. For a framed key equation, see `eq-box` in Section 5.

```
1 #figure(  
2   rect(width: 60%, height: 3em, fill:  
3     colors.primary-lightest),  
4   caption: [The caption, without a  
5     number.],  
6 )  
7 The update rule is  
8 $ \boldsymbol{w} \leftarrow \boldsymbol{w} - \eta \nabla E(\boldsymbol{w}) $
```



The caption, without a number.

The update rule is

$$\boldsymbol{w} \leftarrow \boldsymbol{w} - \eta \nabla E(\boldsymbol{w})$$

8.3 Code

Inline `raw` is the chip of Section 3. Code blocks go through codly: a gray box, gray line numbers, no zebra, never split across pages. The theme re-exports codly, and its `local` wraps codly’s for per-block settings; `#codly-range(..)` and `#no-codly[..]` work as usual. The languages with a label are `python`, `typst`, `bash` and `text` (labelled “saída”).

```

1 == Code blocks
2
3 ```python
4 def step(w, grad, lr=0.1):
5     return w - lr * grad
6 ```
7
8 #local(display-name: true,
9 highlights: ((line: 2, fill:
10 colors.secondary-lighter),))[
11 ```python
12 for epoch in range(10):
13     w = step(w, grad(w))
14 ```
15 ]
16
17 #local(number-format: none)[
18 ```bash
19 typst compile slides.typ
20 ```
21 ]

```

Code blocks

```

1 def step(w, grad, lr=0.1):
2     return w - lr * grad

```

```

1 for epoch in range(10):
2     w = step(w, grad(w))

```

typst compile slides.typ

Ufac • PPGE018 • NEURAL NETWORKS

PROF. AUTHOR NAME 1

local()

```

local(
  body content
  ..args any
) -> content

```

Per-block code settings: codly's `local`, re-exported by the theme so that its code box and `raw` handling also apply inside (this `local` shadows codly's). Typical uses: `#local(number-format: none)[...]` hides the line numbers, `#local(display-name: true)[...]` shows the language label and `#local(highlights: ((line: 2, fill: colors.secondary-lighter),))[...]` highlights a line. codly's `codly-range` and `no-codly` need no wrapper.

Argument

body content

Content with the code blocks.

Argument

..args any

Any argument of codly's `codly` function.

code-languages

dictionary

Languages known to codly in code blocks. The `name` is what `#local(display-name: true)[...]` shows as a label; `text` is labelled "saída" (output), in Portuguese like the other user-facing defaults of the theme.

8.4 Icons

icon()

```
icon(
  name str
  color: auto auto color
  size: 1em length
  baseline: 25% relative
) -> content
```

An icon of the Octicons set (320 names), already wrapped in a `box` for inline use. An unknown name is a compile error.

```
1 #icon("check-circle", color:
  colors.quaternary) Done,
2 #icon("alert") attention and a big
3 #icon("rocket", size: 1.6em, color:
  colors.primary)
```

✔ Done, ⚠ attention and a big 🚀

Argument

name

str

Name of the icon, e.g. "check", "alert", "light-bulb", "mark-github".

Argument

color: auto

auto | color

Fill color. With `auto` it follows the context like bold text: white inside a pill (a `==` subtitle, an exercise header, the title of an `emph-box()`), the box or title color inside `emph-box()` and `==`, black elsewhere.

Argument

size: 1em

length

Width and height of the icon.

Argument

baseline: 25%

relative

Baseline of the box; the default aligns the icon with the text.

```
1 #emph-box(color: colors.tertiary)[
2   #icon("alert") takes the box color.
3 ]
```

⚠ takes the box color.

The 320 names, from [Octicons](#):

accessibility	accessibility-inset	alert	alert-fill	apps
archive	arrow-both	arrow-down	arrow-down-left	arrow-down-right
arrow-left	arrow-right	arrow-switch	arrow-up	arrow-up-left
arrow-up-right	beaker	bell	bell-fill	bell-slash
blocked	bold	book	bookmark	bookmark-slash
briefcase	broadcast	browser	bug	cache
calendar	check	check-circle	check-circle-fill	checkbox
checklist	chevron-down	chevron-left	chevron-right	chevron-up
circle	circle-slash	clock	clock-fill	cloud

	cloud-offline		code		code-of-conduct		code-review		code-square
	codescan		codescan-checkmark		codespaces		columns		command-palette
	comment		comment-discussion		container		copilot		copilot-error
	copilot-warning		copy		cpu		credit-card		cross-reference
	dash		database		dependabot		desktop-download		device-camera
	device-camera-video		device-desktop		device-mobile		devices		diamond
	diff		diff-added		diff-ignored		diff-modified		diff-removed
	diff-renamed		discussion-closed		discussion-duplicate		discussion-outdated		dot
	dot-fill		download		duplicate		ellipsis		eye
	eye-closed		feed-discussion		feed-forked		feed-heart		feed-issue-closed
	feed-issue-draft		feed-issue-open		feed-issue-reopen		feed-merged		feed-person
	feed-plus		feed-public		feed-pull-request-closed		feed-pull-request-draft		feed-pull-request-open
	feed-repo		feed-rocket		feed-star		feed-tag		feed-trophy
	file		file-added		file-badge		file-binary		file-code
	file-diff		file-directory		file-directory-fill		file-directory-open-fill		file-directory-symlink
	file-moved		file-removed		file-submodule		file-symlink-file		file-zip
	filter		fiscal-host		flame		fold		fold-down
	fold-up		gear		gift		git-branch		git-commit
	git-compare		git-merge		git-merge-queue		git-pull-request		git-pull-request-closed
	git-pull-request-draft		globe		goal		grabber		graph
	hash		heading		heart		heart-fill		history
	home		horizontal-rule		hourglass		hubot		id-badge
	image		inbox		infinity		info		issue-closed
	issue-draft		issue-opened		issue-reopened		issue-tracked-by		issue-tracks
	italic		iterations		kebab-horizontal		key		key-asterisk
	law		light-bulb		link		link-external		list-ordered
	list-unordered		location		lock		log		logo-gist
	logo-github		mail		mark-github		markdown		megaphone
	mention		meter		milestone		mirror		moon
	mortar-board		move-to-bottom		move-to-end		move-to-start		move-to-top
	multi-select		mute		no-entry		north-star		note
	number		organization		package		package-dependencies		package-dependents
	paintbrush		paper-airplane		paperclip		passkey-fill		paste
	pencil		people		person		person-add		person-fill
	pin		pin-slash		pivot-column		play		plug
	plus		plus-circle		project		project-roadmap		project-symlink
	project-template		pulse		question		quote		read
	redo		rel-file-path		reply		repo		repo-clone
	repo-deleted		repo-forked		repo-locked		repo-pull		repo-push
	repo-template		report		rocket		rows		rss
	ruby		screen-full		screen-normal		search		server
	share		share-android		shield		shield-check		shield-lock
	shield-slash		shield-x		sidebar-collapse		sidebar-expand		sign-in
	sign-out		single-select		skip		skip-fill		sliders
	smiley		sort-asc		sort-desc		sparkle-fill		sponsor-tiers
	square		square-fill		squirrel		stack		star
	star-fill		stop		stopwatch		strikethrough		sun

 sync	 tab-external	 table	 tag	 tasklist
 telescope	 telescope-fill	 terminal	 three-bars	 thumbsdown
 thumbsup	 tools	 tracked-by-closed-completed	 tracked-by-closed-not-planned	 trash
 triangle-down	 triangle-left	 triangle-right	 triangle-up	 trophy
 typography	 undo	 unfold	 unlink	 unlock
 unmute	 unread	 unverified	 upload	 verified
 versions	 video	 webhook	 workflow	 x
 x-circle	 x-circle-fill	 zap	 zoom-in	 zoom-out

9 Touying features

The theme sits on Touying and leaves its features alone. Animations work on every element of the theme: covered content stays faintly visible under a veil in the page color.

1 **== Animations**

2

3 First this.

4

5 #pause

6

7 > Then a box, ***veiled*** until

8 the next step.

9 #pause

10

11 -> and a list

Animations

First this.

Then a box, veiled until the next step.

→ and a list

Ufac

PPGEE018 • NEURAL NETWORKS

PROF. AUTHOR NAME

1

Animations

First this.

Then a box, veiled until the next step.

→ and a list

Ufac

PPGEE018 • NEURAL NETWORKS

PROF. AUTHOR NAME

1

Animations

First this.

Then a box, veiled until the next step.

→ and a list

Ufac

PPGEE018 • NEURAL NETWORKS

PROF. AUTHOR NAME

1

Keep #pause and #uncover / #only in separate paragraphs: within one paragraph Touying reorders the covered part on the first subslide.

Available without any wrapper from the theme:

Feature	Functions
Animations	#pause, #meanwhile, #jump, #uncover, #only, #alternatives, #item-by-item, #effect, #touying-raw

Feature	Functions
Notes and handouts	<code>#speaker-note</code> (it attaches to the previous slide), <code>#handout-only</code> , <code>config-common(handout: true)</code>
Layout	<code>#slide(composer: (lfr, 2fr))</code> , <code>#cols</code> , <code>components.adaptive-columns</code> , <code>components.full-width-block</code> , <code>utils.fit-to-width</code> , <code>utils.fit-to-height</code>
Overflow	<code>config-common(breakable: false, clip:, detect-overflow:)</code>
Reuse	<code>#touying-recall</code> , <code>touying-get-config</code> , <code>components.progressive-outline</code>

Slides of other Touying themes that this one does not have: `outline-slide`, `focus-slide`, `ending-slide` and `matrix-slide`.

10 Index of definitions

The reference of every function and variable sits in the chapter that introduces it; it is generated from the doc-comments of the package with [tidy](#). Names that start with an underscore are internal and left out.

ufac.typ

empty-slide() p. 12	example-slide() p. 9	exercise-slide() p. 8
new-section-slide() p. 13	slide() p. 11	title-slide() p. 4
ufac-theme() p. 3		

components.typ

arrows() p. 17	cols() p. 22	emph-box() p. 19
eq-box() p. 20	icon() p. 26	local() p. 25
primary() p. 15	quaternary() p. 16	quote-box() p. 21
secondary() p. 15	tertiary() p. 16	

constants.typ

code-languages p. 25	colors p. 23	ufac-logo p. 23
--------------------------------------	------------------------------	---------------------------------